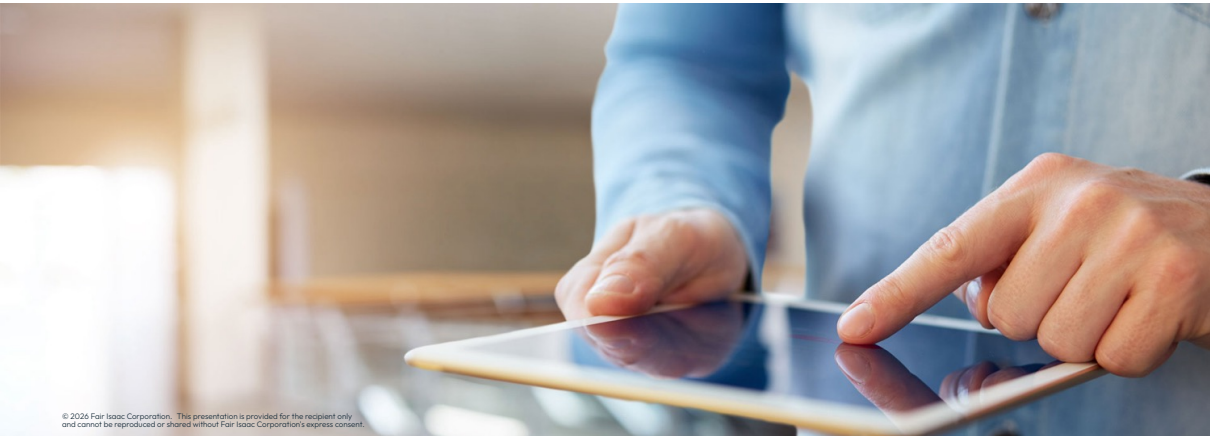
Deployment

Xpress Optimization, FICO



Deploying Mosel models

Embedding models in applications

- Application design considerations
- Deployment options for Mosel models
- Standalone executable
- The Mosel API

Application design considerations

- Model development: once deployed, models often remain in use for periods >10 years ⇒ code needs to be suitable for easy maintenance by changing owners
  - conventions in Mosel model development guidelines for optimization projects document:
    - model code structure and formating
    - versioning
    - naming standards
    - model debugging
    - error handling
    - documentation

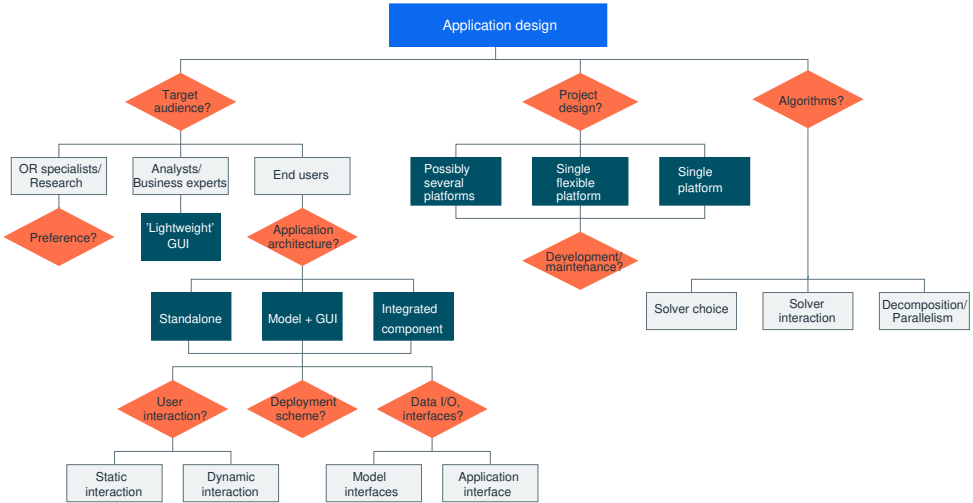
Application design considerations

- Model development: once deployed, models often remain in use for periods >10 years ⇒ code needs to be suitable for easy maintenance by changing owners
  - conventions in Mosel model development guidelines for optimization projects document:
    - model code structure and formating
    - versioning
    - naming standards
    - model debugging
    - error handling
    - documentation
- Comments are essential!

Application design considerations

- Model formulation:
  - Solving time varies considerably among problem instances
    - consider trade-offs between decision type and performance required
  - Do not overmodel!
    - overmodeling can be expensive and useless, depending on use-case
  - Consider end-user involvement from the beginning
    - what is the level/type of interaction required for the end-user?
  - Prefer lightweight / modular integration of model
    - be prepared for future changes and upgrades

Application design considerations



Deployment options for Mosel models

- Distribution as BIM file
- Standalone executable
- Embedding in an application via the Mosel API
  - available for C/C++, Java, .NET and VBA
- Xpress Insight application

Reminder: Compiling Mosel models

- Mosel command line:

```
mosel comp mymodel.mos
mosel comp -g mymodel.mos -o mybim.bim
```

- Mosel language (*mmjobs*):

```
compile("mymodel.mos")
compile("g", "mymodel.mos", "mybim.bim")
```

- Mosel libraries (*e.g.* Java):

```
XPRM mosel; XPRMmodel model;
model = mosel.compile("mymodel.mos");
model = mosel.compile("g", "mymodel.mos", "mybim.bim");
```

Reminder: Compiling Mosel models

- Workbench:
  - select menu *Run* ≫ *Build*

Standalone executable

- *deploy* module: generates an executable containing the BIM file
  - on all supported platforms (generating executables requires C compiler)
  - retrieves arguments from the command line
  - can include additional files (default application data, public keys, certificates)
- Usage (requires C compiler):

```
mosel comp mymodel.mos –o deploy.exe:runmymodel
```

- Output source file `runmymodel.c`:

```
mosel comp mymodel.mos –o deploy.csrc:runmymodel
```

Standalone executable

- *mosdeploy.mos* tool (see XPRESSDIR/examples/mosel/Programming):
  - exploiting *deploy* functionality to generate executables without need for a C compiler by embedding the BIM file into a shell script

Standalone executable

- *mosdeploy.mos* tool (see XPRESSDIR/examples/mosel/Programming):
  - exploiting *deploy* functionality to generate executables without need for a C compiler by embedding the BIM file into a shell script
  - usage examples:

```
mosel mosdeploy -- myfile.mos             creates .bat or .sh file (system-dependent)
mosel mosdeploy -- -cmd myfile.mos        creates .cmd file
mosel mosdeploy -- -o runfile myfile.mos  use output name 'runfile'

mosel mosdeploy -- -exe mosdeploy.mos     creates executable (requires C compiler)
Same as:   mosel comp mosdeploy.mos -o deploy.exe:mosdeploy
mosdeploy myfile.mos                      creates .bat or .sh file
```

Standalone executable

- Retrieving parameter values from the command line
  - replace the parameters block:

```
parameters
  LIMIT=2000
end-parameters
```

by the retrieval of argument value(s):

```
declarations
  LIMIT: integer
end-declarations

if argc>1 then    ! 1: name of program, all following are arguments
  LIMIT:=integer(argv(2))
else
  LIMIT:=200      ! Default value
end-if
```

Standalone executable

- Usage:
  - Compilation (requires C compiler):

```
mosel comp primedeploy.mos –o deploy.exe:runprime
```

  - Execution:
    - Windows:

```
runprime.exe 500
```

    - Unix:

```
runprime 500
```

Standalone executable

- Usage:
  - Compilation (requires C compiler):

```
mosel comp primedeploy.mos –o deploy.exe:runprime
```

  - Execution:
    - Windows:

```
runprime.exe 500
```

    - Unix:

```
runprime 500
```

  - Compilation using *mosdeploy* tool:

```
mosel mosdeploy.mos -- –o runprime primedeploy.mos
```

  - Execution:

```
runprime 500
```

Standalone executable

- Usage:
  - Compilation (requires C compiler):

```
mosel comp primedeploy.mos –o deploy.exe:runprime
```

  - Execution:
    - Windows:

```
runprime.exe 500
```

    - Unix:

```
runprime 500
```

  - Compilation using *mosdeploy* tool:

```
mosel mosdeploy.mos -- –o runprime primedeploy.mos
```

  - Execution:

```
runprime 500
```

  - The model can also be run directly as a BIM file:

```
mosel primedeploy -- 500
```

Project work [P-10]: Standalone executable

- Generate an executable for a Mosel model of your choice:
  - transform any runtime parameters as to read their values from the command line
  - if you do not have any C compiler installed:
    - generate the C source instead of the executable, or
    - work with the *mosdeploy* tool (copy the file XPRESSDIR/examples/mosel/Programming/mosdeploy.mos into your working directory)

What is the Mosel API?

- The Mosel language allows you to formulate optimization problems, and develop optimization methods (*i.e.*, use the Optimizer to solve them), as a Mosel model
- The Mosel API (also Mosel libraries) allows you to embed Mosel models in an application

Programming environments

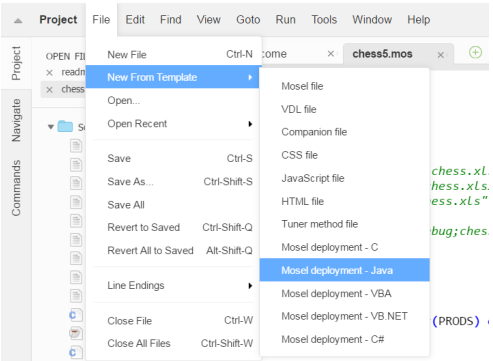
- The Mosel API is available for C/C++, Java, .NET, Python and VBA
- We use Java in the slides, but the functionality applies to all languages, and similar applications can be developed in other languages

Mosel libraries

- Model Compiler Library
  - compiles to a virtual machine
  - binary format architecture independent
- Runtime Library
  - load and run binary (models)
  - access to Mosel internal database (data, solution values, ...)

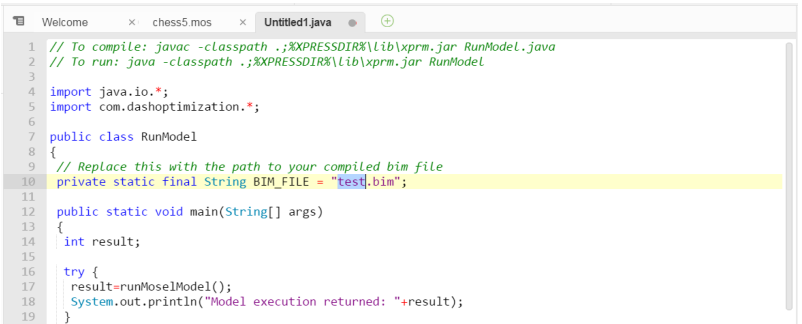
Generating a deployment template

- With Xpress Workbench:
  - compile your model in Xpress Workbench
  - open the menu *File* ≫ *New From Template* then choose the appropriate Mosel deployment file template



Generating a deployment template

- Find the constant with the value `test.bim` near the top of the file and change its value to the name of your BIM file



Generating a deployment template

- Save the file then copy this source file and your BIM file into your project in Microsoft Visual Studio, Eclipse IDE or other development environment
  - Xpress Workbench cannot compile or run source files written in C, C#, Java, VB.NET or VBA; to compile and run the generated source file, use an IDE tool which supports the corresponding language

Mosel library functions

- General:

```
XPRM(), XPRM.getVersion, XPRM.license, ...
```

- Model handling:

```
XPRM.compile, XPRM.loadModel, XPRMModel.run, XPRMmodel.getResult,
XPRMModel.getExecStatus, XPRMModel.reset, ...
```

- Solution information:

```
XPRMModel.getObjectiveValue, XPRMModel.getProblemStatus,
XPRMMPVar.getSolution, XPRMLinCtr.getActivity, ...
```

Mosel library functions

- Accessing model objects:

```
XPRMModel.findIdentifier
```

- Arrays:

```
XPRMArray.getDimension, XPRMArray.getIndexSets,
XPRMArray.getFirstIndex, XPRMArray.nextIndex, XPRMArray.get, ...
```

- Sets:

```
XPRMSet.getSize, XPRMSet.getFirstIndex, XPRMSet.isFixed, ...
```

- Handling of modules:

```
XPRM.findModule, XPRM.setModulesPath, XPRMModule.parameters, ...
```

Project work [C-5]:
Model deployment

- Use Workbench to generate a Java program that compiles and runs model `chess5.mos`
- Modify the program so that the model execution uses the data file `chess5.dat`.
- Check the problem status and output the objective value.

Extending the example

- Retrieving detailed solution information and model data

```
XPRMModel model;
XPRMSet prods;
XPRMArray profit, ax;
XPRMMPVar x;
int[] idx = new int[1];
double val;

// Retrieve solution values and problem data
prods = (XPRMSet)model.findIdentifier("PRODS");
profit = (XPRMArray)model.findIdentifier("PROFIT");
ax = (XPRMArray)model.findIdentifier("x");
// Get the first entry of array 'ax'
// (we know that the array is dense and has a single dimension)
idx = ax.getFirstIndex();
do {
  x = ax.get(idx).asMPVar();      // Get a variable from 'ax'
  val = profit.getAsReal(idx);    // Get the corresponding value
  System.out.println(prods.get(idx[0]) + ": " + x.getSolution() +
                     "\t (profit: " + val + ")");       // Print the solution value
} while(ax.nextIndex(idx));       // Get the next index
```

Extending the example

- Data exchange in memory with host application

```
public class chessio
{
  static int NP = 4;                    // Input data
  static final double[] dur    = {3, 2, 2, 3};
  static final double[] wood   = {1, 2, 3, 6};
  static final double[] profit = {5,12,20,40};
                                        // Array for solution values
  static double[] solution = new double[NP];

  public static void main(String[] args) throws Exception
  {
    int result;
    XPRMModel model;
    XPRM xprm;
```

Extending the example

```
    xprm = new XPRM();                  // Initialize Mosel
    xprm.compile("chess5ioj.mos");      // Compile + load model
    model = xprm.loadModel("chess5ioj.bim");
    xprm.bind("DUR", dur);              // Associate Java objects with
    xprm.bind("WOOD", wood);            // names in Mosel
    xprm.bind("PROFIT", profit);
    xprm.bind("xsol", solution);
    model.execParams = "NP="+NP;        // Set runtime parameters
    model.run();                        // Run the model
    if (model.getProblemStatus()==model.PB_OPTIMAL)
    {               // Check problem status and display the solution
      System.out.println("Objective: " + model.getObjectiveValue());
      for(int i=0;i<NP;i++)
        System.out.println("x(" + (i+1) + "): " + solution[i] +
                           "\t (profit: " + profit[i] + ")");
    }
    model.reset();
  }
}
```

Deploying to Excel

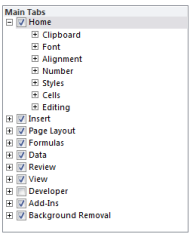
- Generate VBA code via the Workbench deployment template
  - menu *File* ≫ *New From Template*, select *Mosel deployment - VBA*
  - find the constant with the value `test.bim` near the top of the file and change its value to the name of your BIM file
  - copy the displayed VBA code into the clipboard
- Generate the BIM file
- Create a macro-enabled spreadsheet (.xlsm)

Deploying to Excel

- Select Excel menu *Developer* ≫ *Insert*
  - select the button object and position it using the mouse
  - assign a new macro to the button and paste the VBA code from clipboard into the macro
- Using Excel menu *Developer* ≫ *Visual Basic* ≫ *File* ≫ *Import File...* add `xprm.bas` from the `include` subdirectory of Xpress to the project
- Select button with right mouse key to edit its text

Deploying to Excel

- If you do not see the *Developer* menu:
  - Excel 2007: click on the round button top left  then select *Excel Options*; under the *Popular* section, check the option *Show Developer tab*
  - Excel 2010 and newer: on the *File* tab, choose *Options*, then choose *Customize Ribbon* and in the list of *Main Tabs*, select the *Developer* check box:



Output redirection to Excel

- Output redirection to file:
  - Edit the macro (after XPRMinit):

```
Call XPRMsetdefstream(0, XPRM_F_ERROR, ActiveWorkbook.path & "\err.txt")
Call XPRMsetdefstream(0, XPRM_F_WRITE, ActiveWorkbook.path & "\log.txt")
```

- Output redirection to spreadsheet:
  - Edit the macro:

```
nReturn = XPRMsetdefstream(0, XPRM_F_WRITE, XPRM_IO_CB(AddressOf OutputCB))
```

Output redirection to Excel

- Overwriting the same line
  - use `Long` or `LongPtr` depending on VBA version (`LongPtr` for recent/64bit)

```
Public Function OutputCB(ByVal model As LongPtr, ByVal info As LongPtr,
                         ByVal msg As String, ByVal size As Long) As Long
  ' Process windows messages so excel gui responds
  DoEvents

  ' Strip the extra newline character and print to sheet
  Worksheets("Sheet1").Cells(1, 2) = Mid(msg, 1, Len(msg) - 1)
End Function
```

Output redirection to Excel

- Multi-line output:

```
Public Function OutputCB(ByVal model As LongPtr, ByVal info As LongPtr,
                         ByVal msg As String, ByVal size As Long) As Long
  ' Process windows messages so excel gui responds
  DoEvents

  ' Strip the extra newline character and print to sheet
  Worksheets("Sheet1").Cells(ROWNUM, 2) = Mid(msg, 1, Len(msg) - 1)
  Debug.Print Mid(msg, 1, Len(msg) - 1)
  ROWNUM = ROWNUM + 1
End Function
```

- Declare and initialize counter `ROWNUM`:

```
Global ROWNUM As Integer
ROWNUM = 1
```

Output redirection to Excel

- Use Mosel's standard functionality for data input and output from/to Excel
  - create named cell ranges in the spreadsheet and use `initializations to` / `initializations from` in the Mosel model

Summary

- Mosel libraries allow you to embed model programs directly in your application
- Access the solution directly in your application, as alternative to using ODBC
- Enjoy benefits of structured modeling language and rapid deployment when building applications

Summary

- May choose to work with compiled models rather than model source files – provides protection against the user viewing / changing the model
- Compiled models are platform independent

Reference material

- You will find it helpful to refer to the Mosel Libraries Reference Manual
- The part 'Working with the Mosel libraries' of the Mosel User Guide documents examples for different programming language interfaces

Application development with Xpress Insight

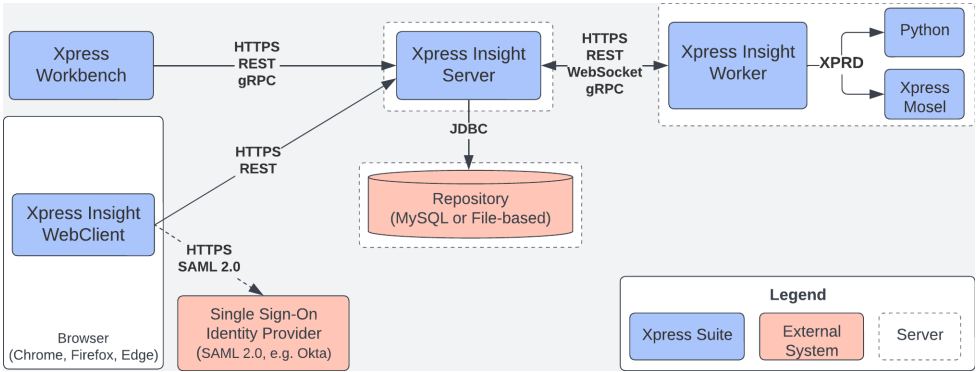
FICO

Xpress Insight

- What is Xpress Insight?
  - Xpress Insight is an extensible application for deploying optimization models
  - client-server architecture supporting different types of clients
  - features include
    - data management (files, attachments, scenarios)
    - model execution
    - scenario analysis
    - drag & drop UI creation
    - visualization with complex business workflows
    - security & access management (SAML2)
    - user and role management (authorities)
    - collaborative platform

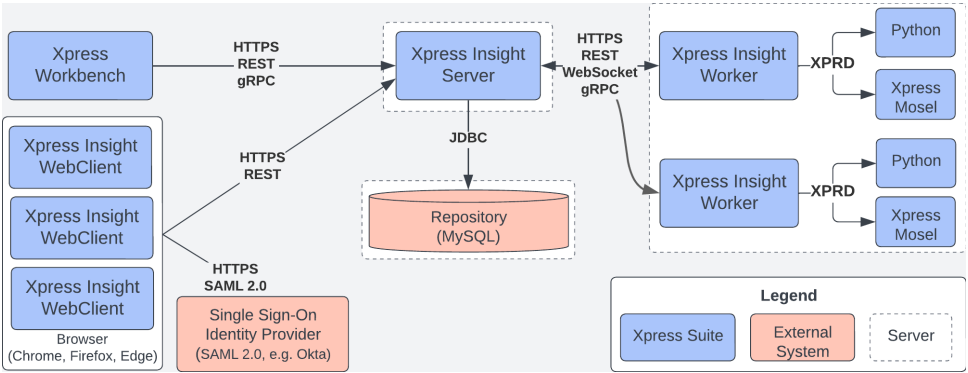
Xpress Insight

- Desktop installation



Xpress Insight

- Multi-user



Xpress Insight: typical workflow

- Load scenario: read input data provided in app archive

Xpress Insight: typical workflow

- Load scenario: read input data provided in app archive
- Modify data: User can change the inputs through the GUI

Xpress Insight: typical workflow

- Load scenario: read input data provided in app archive
- Modify data: User can change the inputs through the GUI
- Run scenario: Execute the Mosel model with current data

Xpress Insight: typical workflow

- Load scenario: read input data provided in app archive
- Modify data: User can change the inputs through the GUI
- Run scenario: Execute the Mosel model with current data
- Analyze the solution

Preparing the model file

```
 uses "mmxprs", "mminsight"
 parameters
   DATAFILE='chess5ins.dat'
 end-parameters

!@insight.manage=input
 public declarations
   PRODS: range                              ! Index set for products
   RESOURCES: set of string                  ! Index set for resources
   PRODNAME,PROFIT: array(PRODS) of string   ! Product data
   RESUSE: array(RESOURCES, PRODS) of real   ! Resource usage coefficients
   RESLIM: array(RESOURCES) of real          ! Resource availability
 end-declarations

!@insight.manage=result
 public declarations
   produce: array(PRODS) of mpvar            ! Array of decision variables
   LimCtr: array(RESOURCES) of linctr        ! Limit on resource usage constr.
   TotalProfit: linctr                       ! Objective function
 end-declarations
```

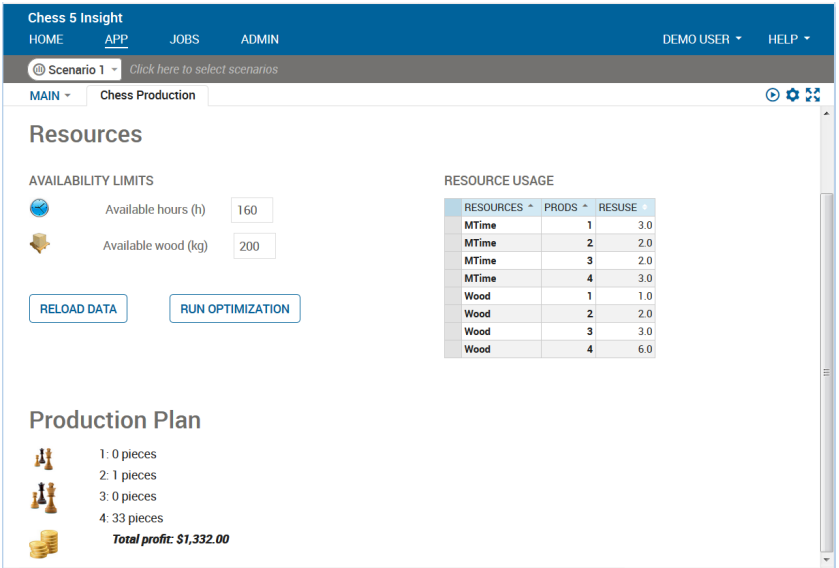
Preparing the model file

```
! Handling model execution modes
 case insightgetmode of
   INSIGHT_MODE_LOAD, INSIGHT_MODE_NONE: do
       initializations from DATAFILE        ! Read data from file
         RESUSE [PRODNAME,PROFIT] as "PRODDATA" RESLIM
       end-initializations
       if insightgetmode=INSIGHT_MODE_LOAD then exit(0); end-if
     end-do
   INSIGHT_MODE_RUN:
     insightpopulate
   else
     writeln("Unknown run mode")
     exit(1)
 end-case

! ... state the problem ...

! Problem solving via Insight
 insightmaximize(TotalProfit)
```

Deployment from Workbench

- Click the 'publish to Insight' button once your model is ready
  - after entering your Insight credentials (default for desktop installation: `admin / admin123`) follow the link to the app loaded in the Insight Web Client
- Workbench also supports the editing of Insight app configuration files
  - VDL view definition (custom web views for editing data or reporting results)
  - XML companion file (project configuration settings)
- To create an Insight app template with the expected subdirectory structure select *Create Xpress Insight Project* at startup (or use menu *Project* ≫ *New Project*)

Xpress Insight Web Client



Reference material

- See section *Deployment to Xpress Insight* of the Getting Started with Xpress manual.
- Xpress Insight documentation: see the Xpress Insight Web Client User Guide and also the Xpress Insight Developer Guide

Thank You

http://www.fico.com/xpress